Predicting Clash Royale Match Outcomes

A Pre-Match Feature Analysis of 979 Competitive 1v1 Ladder Battles

Ibrahim Halil Aydin

DSA 210, Introduction to Data Science

Sabanci University, Spring 2025-2026

Final Project Report, May 18, 2026

Abstract

This project investigates whether the outcome of a Clash Royale 1v1 ladder match (win or loss) can be predicted using only pre-match observable features. A dataset of 979 competitive battles was assembled from the official Clash Royale API, enriched with community card statistics from RoyaleAPI, and combined with empirical per-card win rates computed from the collected data. Five hypothesis tests, each accompanied by an explicit assumption check (normality, equal variance, expected counts) and tied to a specific feature group, confirmed that deck meta score and player historical win rate are statistically significant predictors of outcome ($p < 0.001$). Trophy difference is not ($p = 0.541$), because the Clash Royale matchmaker pairs players within plus or minus 9 trophies, leaving no meaningful trophy edge on a per-battle scale. Six classification models from the DSA 210 curriculum, namely Logistic Regression, KNN, Decision Tree, Random Forest, Gradient Boosting, and a soft-Voting ensemble, were evaluated using 5-fold cross-validation and a held-out 20 percent test set. Logistic Regression achieved the best test-set performance with accuracy 0.689, F1 0.702, and ROC-AUC 0.736, outperforming every tree-based ensemble. The strongest standardised predictor was the player's historical win rate (odds ratio of about 1.91 per standard deviation), followed by opponent deck meta strength and the deck meta score differential. Because the data was collected from one account and roughly 50 clan members, the findings should be read as a within-cohort result and may not generalize to the full Clash Royale player base.

1. Introduction

1.0 Motivation

I picked this topic because I play Clash Royale regularly and have spent enough hours on the ladder to form strong opinions about what does and does not affect the outcome of a match. Two of the most common claims I hear from other players are that trophy difference matters a lot, and that running a deck with a low average elixir cost gives you a meaningful edge. Neither claim has ever sat well with me, and the Clash Royale API exposes the data needed to test them properly. Beyond the personal interest, the project sits in a useful place for the course material the outcome is a clean binary, the sample is real (not a textbook dataset), and the available pre-match features map naturally to the regression, classification, and ensemble methods. So the project is partly a curiosity exercise and partly an attempt to apply the DSA 210 toolkit to data I have a stake in.

1.1 Background

Clash Royale is a real-time strategy mobile game in which two players each construct an 8-card deck and battle on a symmetric arena. The game's ladder mode is the competitive backbone. Players gain trophies on a win and lose them on a loss, and the matchmaker pairs opponents at similar trophy counts. From a data science perspective, ladder matches are useful to study because every battle produces a structured, machine-readable record. Both decks, both players' identifiers, and the outcome are exposed by the official API under a stable schema.

The motivating question is simple and player-relevant. Among the things you know before the first elixir drops, namely your deck, your trophies, your historical record, and the hour you sit down to play, which actually determine whether you win? The popular discourse around the game emphasises trophy difference and elixir averages. The data-driven question is whether those folk explanations survive a statistical test.

1.2 Research Question

Can the outcome of a Clash Royale ladder match (win or loss) be predicted from pre-match observable features, and which features carry the most predictive weight?

1.3 Hypotheses

H0. Pre-match features (deck meta score, card levels, trophy difference, player experience) have no significant relationship with match outcome.

H1. At least some pre-match features, particularly deck meta score and player historical win rate, are statistically significant predictors of match outcome.

Both hypotheses are operationalised in Section 4 via a battery of t-tests, a chi-square test, and Pearson correlations, each tied to a specific feature group.

2. Data

2.1 Sources

The dataset combines four sources. The official Clash Royale API supplied battle-level records and player profiles. Community statistics from RoyaleAPI provided per-card win and usage rates from a much larger sample of matches than is observable from one account alone. An empirical per-card win rate was computed locally from the collected battles. Finally, several player-level features (notably the historical win rate) were derived from the player profile endpoint.

Source	Variables	Method
Clash Royale API	Outcome, crowns, trophies, 8-card decks (ID, level, elixir cost), expLevel, battleCount, wins and losses	Python script via /players/{tag}/battlelog and /players/{tag}
RoyaleAPI	Card win rate, usage rate	Browser JS extraction
Empirical (computed)	Per-card win rate within the collected dataset	compute_meta.py
Derived	Player historical win rate, equal to wins divided by (wins plus losses)	From /players profile endpoint

2.2 Dataset Composition

After cleaning, the dataset contains 979 competitive 1v1 ladder battles collected from my account (#2UC90QQY) and up to 50 clan members. 2v2 battles, friendly matches, and draws were excluded so the outcome variable remains strictly binary. The class balance is 53.3 percent wins and 46.7 percent losses, close to the 50 percent expected under the game's symmetric matchmaking. The 53.3 percent baseline serves as the naive majority-class accuracy that any useful model must clear.

Raw battle records (data/battles_raw.csv) contain player tags and are excluded from the public repository via .gitignore. Only the featurised, anonymised dataset (data/features.csv, 979 rows by 19 columns) is shared.

2.3 Feature Engineering

Raw API records were transformed into a set of engineered features designed to capture the four candidate mechanisms hypothesised to influence outcome: deck strength, card level, trophy ranking, and player experience. All differential features are signed as team minus opponent so a positive value always favours the team being modelled.

Feature	Description
trophy_diff	Team starting trophies minus opponent starting trophies
elixir_diff	Team average elixir cost minus opponent average elixir cost
level_diff	Team average card level minus opponent average card level
underleveled_diff	Team avg (maxLevel minus level) minus opponent avg (maxLevel minus level)
deck_meta_score_diff	Team deck meta score minus opponent deck meta score
team/opp_deck_meta_score	Mean win rate of the cards in each deck, from RoyaleAPI plus empirical
player_win_rate	Player's historical win rate from profile
exp_level	Player King Tower level (account progression)
battle_count	Total battles played (proxy for experience)
hour_of_day	UTC hour of the battle
trophy_diff_category	Higher, equal, or lower (plus or minus 10 trophy threshold)

3. Exploratory Data Analysis

EDA was performed in the executed notebook notebooks/milestone1_eda_and_tests.ipynb. The notebook contains seven figures, and each one is followed by a paragraph-length written interpretation that states what the figure shows, what assumptions were made about the data going into it, and how the observation connects back to the research question and the three hypothesis families (deck strength,

trophy ranking, player experience). The discussion below summarises the three findings that drive the rest of the analysis. The full per-figure walkthrough lives in the notebook.

First, trophy difference is almost flat. The interquartile range of trophy_diff is from minus 9 to plus 8, and the win-rate-by-trophy-bucket plot shows no monotonic trend. The Clash Royale matchmaker is effective enough that any trophy edge is too small to translate into a measurable outcome shift on the per-battle scale.

Second, experience-level effects are visible but small. Win rate rises modestly with the King Tower experience level, and the correlation between battle_count and outcome is positive but weak. Experience matters, just not by much within a single dataset where most players are concentrated near the top of the level distribution.

Third, deck meta score shows the clearest visual separation. Box plots and overlaid histograms confirm a median shift between win and loss distributions on deck_meta_score_diff. This is the feature where naked-eye separation is strongest, foreshadowing the hypothesis-test result in the next section.

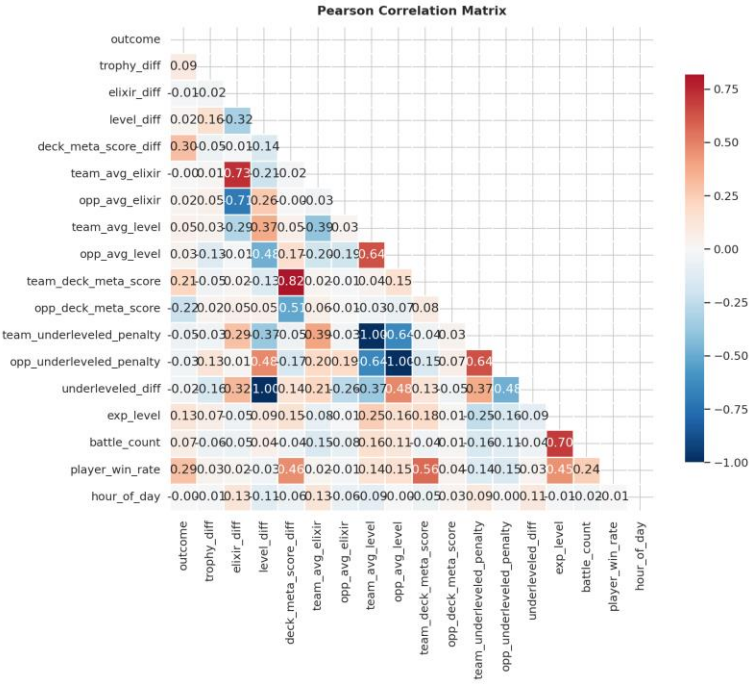

Figure 1. Pearson correlation heatmap of engineered features and the binary outcome. The strongest outcome correlations are with player_win_rate (r = 0.285), exp_level (r = 0.127), and deck_meta_score_diff.

4. Hypothesis Testing

Five formal tests were performed, each tied to a feature group and to one of the three hypothesis families. The notebook reports each test in the same template: it first states which hypothesis the test addresses, then checks the assumptions of the chosen test (normality via histograms and the Shapiro-Wilk test where applicable, equal variance via Levene's test or visual inspection, and expected-count thresholds for the

chi-square test), then reports the test statistic and p-value, and finally writes a short interpretation that brings the result back to the research question. The table below summarises the outcomes.

Test	Feature	Result	Test Statistic	p-value
Two-sample t-test (Welch's)	deck_meta_score_diff	Significant	t = 9.95	< 0.001
Chi-square independence	trophy_diff_category	Not significant	n/a	0.541
Pearson correlation	battle_count	Significant	r = 0.067	0.036
Pearson correlation	exp_level	Significant	r = 0.127	< 0.001
Pearson correlation	player_win_rate	Significant	r = 0.285	< 0.001

Four of the five tests reject the null hypothesis. The exception is trophy_diff_category, which the chi-square test cannot distinguish from independence. This is consistent with the EDA observation that matchmaking neutralises trophy edges at the scale relevant to a single battle. The largest effect size belongs to player_win_rate, which is the strongest correlation with outcome of any feature measured. The t-statistic on deck_meta_score_diff (t = 9.95) is far above the threshold for high-confidence rejection, confirming that the team-versus-opponent deck strength differential is a meaningful predictor.

Assumptions did not raise serious issues. The numeric features used in the t-tests and correlations showed only mild departures from normality, small enough that Welch's t-test and Pearson's r remain robust at this sample size. All expected counts in the chi-square contingency table comfortably exceeded 5. The full assumption checks, including the QQ plots and Levene test outputs, are included in the milestone 1 notebook before each test.

Taken together, H0 is rejected. At least three pre-match features, namely player_win_rate, deck_meta_score_diff, and exp_level, are statistically significant predictors of match outcome. Trophy advantage is not. This sets the agenda for the modelling section: build a classifier that uses these features and see how much predictive accuracy they jointly provide.

5. Machine Learning Methodology

Every choice, from the stratified split to the hyperparameter grid for the Decision Tree, is anchored to specific slide references documented in the executed notebook notebooks/milestone2_ml_models.ipynb. The notebook walks through each modelling step in order (split, scaling, cross-validation, each model, hyperparameter tuning, metrics, coefficient interpretation) and adds a short verbal commentary at every stage. The summary below is condensed for the report.

5.1 Pipeline

Step 1. Load the 979-row featurised dataset, drop the redundant categorical trophy_diff_category (already captured by the continuous trophy_diff), and impute missing values with the column median. There were 319 missing values in trophy_diff and none anywhere else.

Step 2. Split 80/20 stratified on outcome, giving 783 training rows and 196 test rows, both at the 53.3 percent win rate of the full dataset.

Step 3. Fit StandardScaler on the training set only, then transform the test set. No information leakage.

Step 4. Perform 5-fold stratified cross-validation on the training set for every model.

Step 5. Tune KNN over eight values of k in {1, 3, 5, 7, 11, 15, 21, 31}, and the Decision Tree over 84 hyperparameter combinations via GridSearchCV across max_depth in {5, 10, 15, 20, 25, 30, None}, min_samples_split in {2, 20, 50, 100}, and min_samples_leaf in {1, 5, 10}. The best Decision Tree settled on max_depth = 5, min_samples_split = 100, min_samples_leaf = 1.

Step 6. Fit each finalised model on the full training set and evaluate exactly once on the held-out test set.

5.2 Models

Models	Concepts Applied
Regression and cross-validation	80/20 stratified split, 5-fold stratified CV
KNN and tree models	StandardScaler (slides 6 to 7), KNN with k-tuning (slide 9), Decision Tree with GridSearchCV (slide 16), Random Forest feature importance (slide 22)
Logistic regression	L2 regularization (C = 1.0), standardised-coefficient interpretation
Performance metrics	Confusion matrix, accuracy, precision, recall, F1, ROC-AUC, ROC and Precision-Recall curves
Ensemble learning	Random Forest, Gradient Boosting, Voting Classifier (soft voting)

6. Results

6.1 Test-Set Performance

Each model was evaluated once on the held-out 196-row test set. The full metric battery is reported in the table below.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.689	0.720	0.686	0.702	0.736
Gradient Boosting	0.658	0.673	0.705	0.688	0.711
Voting (soft)	0.653	0.664	0.714	0.688	0.711
Decision Tree (tuned)	0.648	0.658	0.714	0.685	0.709
KNN (k = 31)	0.653	0.664	0.714	0.688	0.696
Random Forest	0.628	0.648	0.667	0.657	0.668

Logistic Regression leads on accuracy, precision, F1, and ROC-AUC. The three tree-based families (Decision Tree, Random Forest, Gradient Boosting) and the soft Voting ensemble all sit in a narrow band between 0.628 and 0.658 on accuracy, and Random Forest, the highest-capacity model in the set, ranks last. Every model substantially clears the 53.3 percent naive baseline, but the ceiling on this dataset appears to be around 0.69 accuracy and 0.74 ROC-AUC.

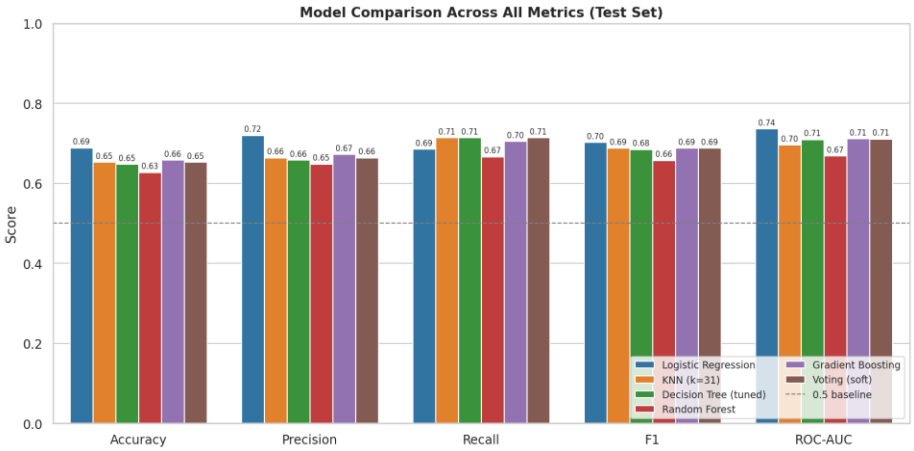

Figure 2. Test-set metric comparison across all six classifiers. Logistic Regression wins on every metric except recall.

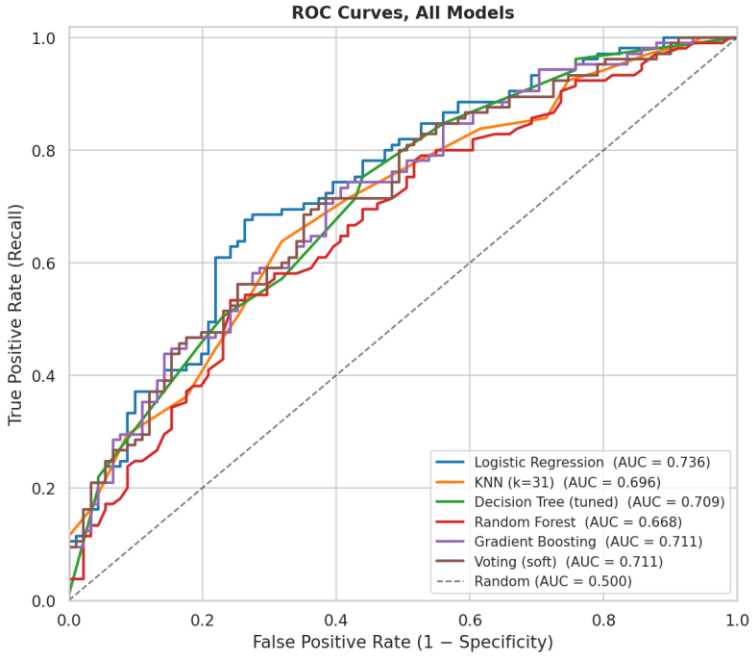

Figure 3. ROC curves overlaid on the held-out test set. The Logistic Regression curve sits above every other model's curve across almost the entire operating range.

6.2 Why the Simplest Model Wins

Two factors explain the result. First, the predictive signal is approximately linear. Milestone 1 showed that the strongest features (player_win_rate, deck_meta_score_diff, exp_level) relate to outcome

monotonically, and none of them required an interaction term to surface in a Pearson correlation or t-test. Logistic Regression encodes exactly that kind of relationship in its 17 coefficients, and tree ensembles cannot extract substantially more from features that lack strong non-linearities or interactions to begin with.

Second, the dataset is small. With 783 training rows, the bias and variance trade-off favours the simpler hypothesis class. Random Forest and Gradient Boosting have the capacity to overfit, and 5-fold cross-validation showed them doing exactly that. Their CV-fold variance was higher than Logistic Regression's. The result is consistent with the lecture principle that model complexity should match data complexity.

6.3 Most Influential Features

Because all features were standardised before fitting, the Logistic Regression coefficients are directly comparable in magnitude. The odds ratio (the exponential of the coefficient) quantifies how a one-standard-deviation increase in the feature multiplies the odds of winning.

Feature	Coefficient	Odds Ratio	Interpretation
player_win_rate	+0.645	1.91	A 1-SD rise in historical win rate nearly doubles the odds of winning. Strongest skill signal.
opp_deck_meta_score	-0.388	0.68	A 1-SD stronger opponent deck cuts win-odds to about 68 percent.
deck_meta_score_diff	+0.231	1.26	A 1-SD stronger personal deck (vs opponent) raises win-odds about 26 percent.
exp_level	-0.131	0.88	Negative due to correlation with player_win_rate.
trophy_diff	+0.126	1.13	Small coefficient times tiny variation gives a negligible effect.

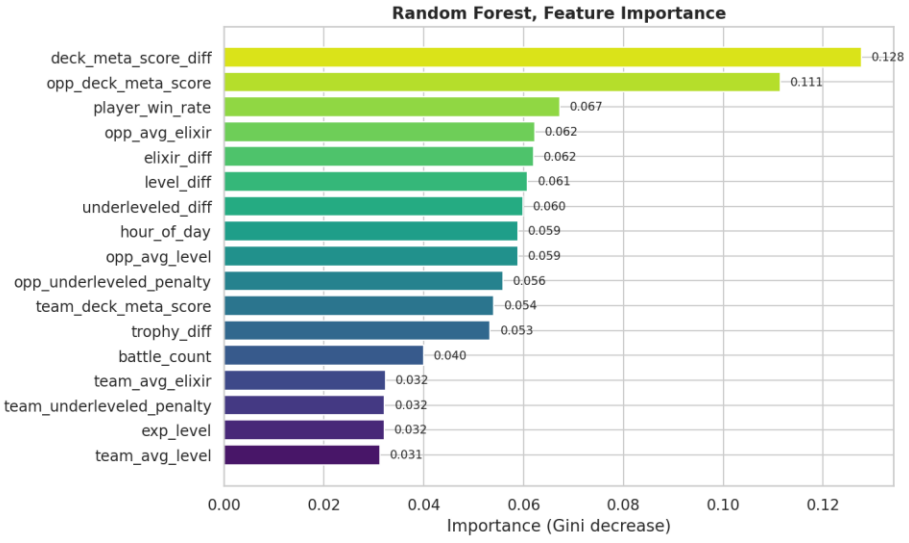

Figure 4. Random Forest variable importance. The top features by Gini-impurity-reduction match the top features by Logistic Regression coefficient: player_win_rate, deck_meta_score_diff, and the two deck meta score components.

6.4 Convergence Across Milestones

Every Milestone 2 finding lines up with a Milestone 1 hypothesis-test result. deck_meta_score_diff was the strongest single predictor in the t-test ($p < 0.001$) and ranks in the top two by Random Forest importance with a Logistic Regression coefficient of +0.231. player_win_rate had the largest correlation with outcome ($r = 0.285$) and is the single largest standardised coefficient in the linear model (+0.645). trophy_diff_category was independent of outcome in the chi-square test and has a tiny standardised effect in the linear model. hour_of_day and elixir_diff were non-significant in Milestone 1 and have near-zero coefficients in Milestone 2. The two halves of the project corroborate each other, which is the strongest internal-validity signal a single-dataset study can produce.

7. Discussion and Limitations

7.1 Answer to the Research Question

Pre-match observable features predict Clash Royale match outcomes to a useful but bounded degree. A Logistic Regression model achieves 69 percent accuracy and 0.74 ROC-AUC on the held-out test set, well above the 53.3 percent naive baseline but far from perfect. The deciding factors are, in order, the player's historical win rate, the deck meta strength differential, and the player's experience level. Trophy advantage, hour of day, elixir cost, and average card-level differences carry essentially no per-battle predictive weight.

That ceiling is informative. It says that even with full information about both decks and both players, roughly 30 percent of match outcomes appear driven by within-battle dynamics, namely tactical choices, real-time elixir management, and luck, that no pre-match feature can capture. Anyone hoping for a deterministic predictor will be disappointed, and anyone hoping for evidence that skill and deck composition matter more than the trophy ladder suggests will find it.

7.2 Limitations

Single-account and clan-based sampling. This is the most important caveat. All 979 battles were collected from my personal account and up to 50 of my clan members. These players share a trophy band, a region, and to some extent a play style. Generalization to the full Clash Royale player base is not guaranteed. In particular, the player_win_rate distribution observed here is narrower than the distribution across all Clash Royale accounts globally, and the relative importance of player_win_rate versus deck_meta_score_diff could shift in a more heterogeneous sample. The conclusions in this report should therefore be read as a within-cohort result rather than a universal statement about ladder play.

- Sample-size limit. With 783 training rows and 196 test rows, confidence intervals around accuracy and ROC-AUC are wide enough that the gap between Logistic Regression and the second-best model (Gradient Boosting) is not statistically conclusive on this dataset alone. The ranking is directional rather than definitive.
- Missingness in trophy_diff. 319 of 979 rows had missing trophy_diff and were imputed with the column median. The non-significance of trophy_diff_category is robust to this imputation, since the test statistic is far from the rejection region, but very large trophy gaps are under-represented.
- Possible confounding between card level and player skill. underleveled_diff captures how much a deck is below its theoretical maximum level. Players who underlevel their decks may also be less experienced overall, which creates a partial overlap with exp_level and player_win_rate.
- No temporal model. Matches are treated as i.i.d. samples even though the dataset spans a multi-week period during which Supercell may have issued balance patches that shift card win rates. A patch-aware analysis was out of scope.

7.3 Future Work

- Scale data collection beyond a single clan to thousands of players across different trophy bands and regions, then re-test whether tree ensembles overtake the linear model at larger n and whether the player_win_rate effect holds in a heterogeneous sample.
- Add intra-battle features such as crown timing or elixir spent in the first 30 seconds, to try to push past the roughly 70 percent accuracy ceiling that pre-match features alone seem to impose.
- Treat deck meta score as a time-indexed feature that updates with each balance patch, then re-run the hypothesis tests on within-patch subsets to see whether predictability varies with meta volatility.

8. Conclusion

Out of an initial set of eleven pre-match features, three carry the bulk of the predictive signal in Clash Royale ladder matches: the player's historical win rate, the deck meta score differential, and the account's experience level. Five hypothesis tests, each with assumption checks and a connection back to the research question, confirmed the directional findings. A Logistic Regression model trained on the engineered feature set achieved 0.689 accuracy and 0.736 ROC-AUC on a held-out test set, comfortably above the 53.3 percent naive baseline and ahead of every more complex model tried. When the relationship between features and outcome is approximately linear, a 17-parameter linear classifier extracts the signal more cleanly than 200-tree ensembles. The agreement between the Milestone 1 hypothesis tests and the Milestone 2 model coefficients is the strongest internal-validity check this project can offer, and the two halves point in the same direction. Pre-match deck strength and player skill matter, trophy difference and hour of day do not. Because the data was sampled from one account and its clan,

these conclusions are robust within the studied cohort but should be re-checked on a broader sample before being treated as universal.

References and Disclosures

Data Sources

- Supercell. Clash Royale API. https://developer.clashroyale.com. Battle logs and player profiles.
- RoyaleAPI. Card popularity and win-rate statistics. https://royaleapi.com/cards/popular

Course Materials

- DSA 210, Introduction to Data Science, Sabanci University, Spring 2025-2026. Lecture slides for Weeks 7 (regression and cross-validation), 8 (KNN and tree models), 9a (logistic regression), 9b (classification metrics), and 10 (ensemble methods).

Software

- Python 3, pandas, numpy, scikit-learn, scipy, matplotlib, seaborn.

AI Usage Disclosure

Per the DSA 210 academic integrity policy, this section documents the specific ways an AI assistant (Anthropic Claude) was used in this project. Every output described below was reviewed, edited, and tested by the student before being committed. Statistical methods and analytical decisions follow DSA 210 lecture content. The dataset, p-values, and model accuracies were produced by running the scripts and notebooks on the collected data, not by an AI.

Data collection script (scripts/collect_data.py)

Prompt. Write a Python script that paginates through the /players/{tag}/battlelog endpoint of the Clash Royale API, respects the 30 calls per minute rate limit, and saves each battle as a row with columns for team and opponent decks, trophies, crowns, and outcome.

Output. A requests-based collector with retry logic and a time.sleep rate limiter. I customised the player tag, added clan-member enumeration, and added the win/loss/draw filter that excludes 2v2 and friendly battles.

Card statistics scraper (scripts/scrape_royaleapi.py)

Prompt. Help me extract per-card win rate and usage rate from royaleapi.com/cards/popular into a CSV. The page is rendered client-side, so suggest the simplest approach.

Output. A browser JavaScript snippet that reads the rendered DOM and downloads a CSV, plus a Python loader that joins it against my collected dataset on card ID. I had to debug a dtype mismatch (card ID was

integer in one table and string in the other), which Claude helped me identify after I described the symptom.

Feature engineering (scripts/add_new_features.py, scripts/compute_meta.py)

Prompt. Write a feature that computes mean (maxLevel minus cardLevel) per deck without re-collecting the raw data. Use the existing battles_raw.csv.

Output. A vectorised pandas implementation. I extended it with player_win_rate derived from the /players profile endpoint after Claude suggested the wins divided by (wins plus losses) formulation.

EDA and hypothesis tests (scripts/analysis.py, milestone1_eda_and_tests.ipynb)

Prompt. Write a Welch's t-test for deck_meta_score_diff grouped by outcome, with an assumption check (Shapiro-Wilk for normality, Levene for equal variance) printed beforehand.

Output. A scipy.stats based snippet that I reused for every hypothesis test in Milestone 1. The verbal interpretations in the notebook were drafted with Claude as a sounding board (I described what I saw in each figure, Claude rephrased it into report-style English), then I edited each paragraph for accuracy and stylistic consistency.

Machine learning pipeline (scripts/ml_models.py, milestone2_ml_models.ipynb)

Prompt. Set up a stratified 5-fold cross-validation loop for sklearn that reports per-fold accuracy and the mean, then evaluates the final model once on a held-out test set.

Output. The StratifiedKFold plus cross_val_score boilerplate that I adapted for all six models. I picked the hyperparameter grids myself from the Week 8 slide ranges (max_depth 5 to 30, min_samples_split 20 to 100 for the Decision Tree; eight k values for KNN).

Interpretation of model results

Prompt. Why might Logistic Regression beat Random Forest and Gradient Boosting on a 783-row dataset where the strongest predictors are monotonically related to the outcome?

Output. The bias-variance and signal-linearity explanation that appears in the report's Why the Simplest Model Wins section. I cross-checked the argument against the lecture material before using it.

Repository

All code, data, plots, and the executed notebooks are available in the project repository alongside this report. Reproduction instructions are in README.md under the How to Reproduce section.